DÍA MES AÑO NOTA /

HW05 - Summary of objet Oriented Modeling and UML

1) 1.0.10 - Object - oriented Modeling Practices

Theory:
Object- oriented modeling helps developing design software that is easy to understand, maintain, and modify. It represent real-world concepts and improves communication throughout software development

- Encapsulation: Class variables should be private to protect data and control access.

- Conceptual Modeling: Helps understand real-world systems, identify important design decisions, and communicate ideas

- Loosely coupled layers: Organizes software into separate layers, such as user interface, application logic, and peristent storage.

- Aggregation over Inheritance: Aggregation provides flexibility because objects can be connected or changed dynamically

- Program to an Interface: Reduces unnecessary dependencies between clases.

2) 1.05 0t - Introduction to UML

Theory:
UML stands for Unified Modeling Language. It is a visual language used to describe, design, and documents software systems. It helps developers understand system structure and behavior.

- Analysis Perspective: Represents real-word concepts and the problem being studied.

Scanned with CamScanner

DÍA MES AÑO NOTA /

-Specification Perspective: Defines system boundaries and software requirements.

- Implementation Perspective: focuses on software objects, programming languages, and execution details.

- Structural Diagrams: Show the static structure of system, such as class and component diagrams.

- Behavioral Diagrams: Show system behavior and interactions, such as use case, sequence, and activity diagrams.

3) 1.OS.08 - Basic Use Cases

Theory: use cases describe how users or external systems interact with software to achieve specific goals. The focus on user requirements rather than internal implementation details.

- Actors: People, external systems, or devices that interact with the software.

- Use Cases: Explain what the user wants to accomplish and why.

- <<include>>: Represents required behavior that is include in another use case.

- <<extend>>: Adds optional, conditional, or exceptional behavior to a base use case.

Scanned with CamScanner

DÍA MES AÑO NOTA /

4) 105.09 - Class Diagrams

Theory: class diagrams represent classes, objects, attributes, methods, and relationships in a software systeam.

- class : A blueprint for creating objects
- object : An individual instance of a class.
- Generalizations: An inheritance or "is-a" relationship
- Association: A connection between classes.
- Aggregation: A weak whole-part relationship.

5) 1.0S10 - UML Quick Reference Guide

General Overview

Serves as a concise cheat sheet for UML analysis use cases, class diagrams, activities, interactions and state charts.

Specific key Points:

- Activity Diagrams: Track control and object flows using initial, action, fork, and join nodes.
- Interaction Diagram: focus on object communication via communication and sequence diagrams.
- State charts: Model lifecycle states, triggers, guards and actions.

Scanned with CamScanner